Section 1: Streaming Analytics

EECS E6893 Big Data Analytics - Homework 2
Fall 2026
Yihang Sun

This section simulates a live stock-data stream by repeatedly pulling historical AAPL one-minute bars from the Massive API, incrementally maintaining a Spark DataFrame, and updating a separate 30-minute moving-average DataFrame.

1. Streaming Setup and Workflow

The final implementation was run in Google Colab with PySpark. I used AAPL one-minute price data from a completed trading day so the program could simulate a real-time stream without requesting same-day market bars. Each API request covers one hour of data. The simulated window advances by five minutes after each request, and the process runs for seven API calls in total.

The configuration below sets the required seven calls, five-minute spacing, one-hour pull window, and five-minute step. This means the first request covers one hour, while the seventh request ends 30 minutes later, producing about 1.5 hours of cumulative data.

```
# ----------------------------
# Configuration
# ----------------------------
TICKER = "AAPL"
BASE_URL = "https://api.massive.com"
NUM_CALLS = 7
PULL_EVERY_SECONDS = 300       # REQUIRED final run: 300 seconds = 5 minutes
WINDOW_MINUTES = 60
STEP_MINUTES = 5
```

Figure 1. Streaming configuration used for the final run.

Incremental loading

The streamed Spark DataFrame is not built by blindly appending every pull. Consecutive one-hour windows overlap heavily, so appending would create duplicate timestamps. I therefore used a left anti join on Stock Name and UTC Timestamp to remove old rows that overlap with the newest pull. The newest records are then added with unionByName, duplicates are removed, and the DataFrame is ordered by timestamp.

```
def incremental_upsert(existing_df, incoming_df):
    """
    Replace overlapping (Stock Name, UTC Timestamp) rows with the newest pull,
    using the assignment hint: a left anti join.
    """
    keys = ["Stock Name", "UTC Timestamp"]
    incoming_keys = incoming_df.select(*keys).dropDuplicates()

    old_non_overlapping = existing_df.join(
        incoming_keys,
        on=keys,
        how="left_anti",
    )

    return (
        old_non_overlapping
        .unionByName(incoming_df)
        .dropDuplicates(keys)
        .orderBy("UTC Timestamp")
    )
```

Figure 2. Incremental upsert logic using a left anti join and unionByName.

2. 30-Minute Moving-Average Calculation

After each streaming update, the program calculates trailing 30-minute moving averages for close, low, high, open, and volume. The moving-average window is time-based rather than simply using the last 30 rows. This is important because minute-level market data can occasionally have missing timestamps. For each observation, the window includes the current minute and the previous 29 minutes.

```
def compute_30_minute_moving_averages(all_stock_df):
    """
    Compute trailing 30-minute moving averages ending at each observation time.

    The window is time-based rather than "last 30 rows", so a few missing
    minute bars do not break the 30-minute interpretation.
    """
    with_seconds = all_stock_df.withColumn(
        "_event_seconds",
        F.col("UTC Timestamp").cast("long")
    )

    # Current minute + previous 29 minutes = a 30-minute trailing window.
    w = (
        Window
        .partitionBy("Stock Name")
        .orderBy(F.col("_event_seconds"))
        .rangeBetween(-(29 * 60), 0)
    )

    return (
        with_seconds
        .select(
            F.col("UTC Timestamp").alias("Datetime"),
            F.avg("c").over(w).alias("c_MA"),
            F.avg("l").over(w).alias("l_MA"),
            F.avg("h").over(w).alias("h_MA"),
            F.avg("o").over(w).alias("o_MA"),
            F.avg("v").over(w).alias("v_MA"),
        )
        .orderBy("Datetime")
    )
```

Figure 3. Time-based 30-minute moving-average calculation in PySpark.

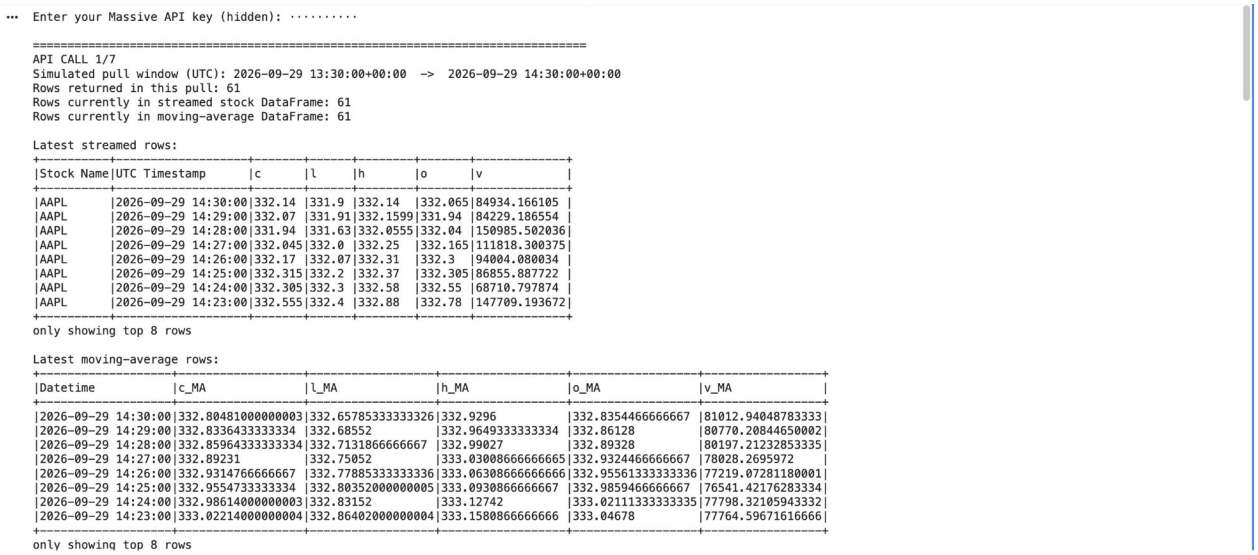
The moving averages are stored in a separate Spark DataFrame. When timestamps are recomputed after a new pull, the newer values replace older versions with the same Datetime key, using the same anti-join-and-union pattern as the streamed stock data.

```python
def incremental_replace_ma(existing_ma_df, fresh_ma_df):
    """
    Keep the moving-average table as a separate incrementally maintained DF.
    Recomputed timestamps replace older versions.
    """
    fresh_keys = fresh_ma_df.select("Datetime").dropDuplicates()
    old_non_overlapping = existing_ma_df.join(
        fresh_keys,
        on="Datetime",
        how="left_anti",
    )
    return (
        old_non_overlapping
        .unionByName(fresh_ma_df)
        .dropDuplicates(["Datetime"])
        .orderBy("Datetime")
    )
```

Figure 4. Incremental maintenance of the moving-average DataFrame.

3. Seven-Call Streaming Simulation

The first API call pulled 61 one-minute rows from 13:30 to 14:30 UTC. At this point, both the streamed stock DataFrame and the moving-average DataFrame contained 61 rows.



```
Enter your Massive API key (hidden): ··········

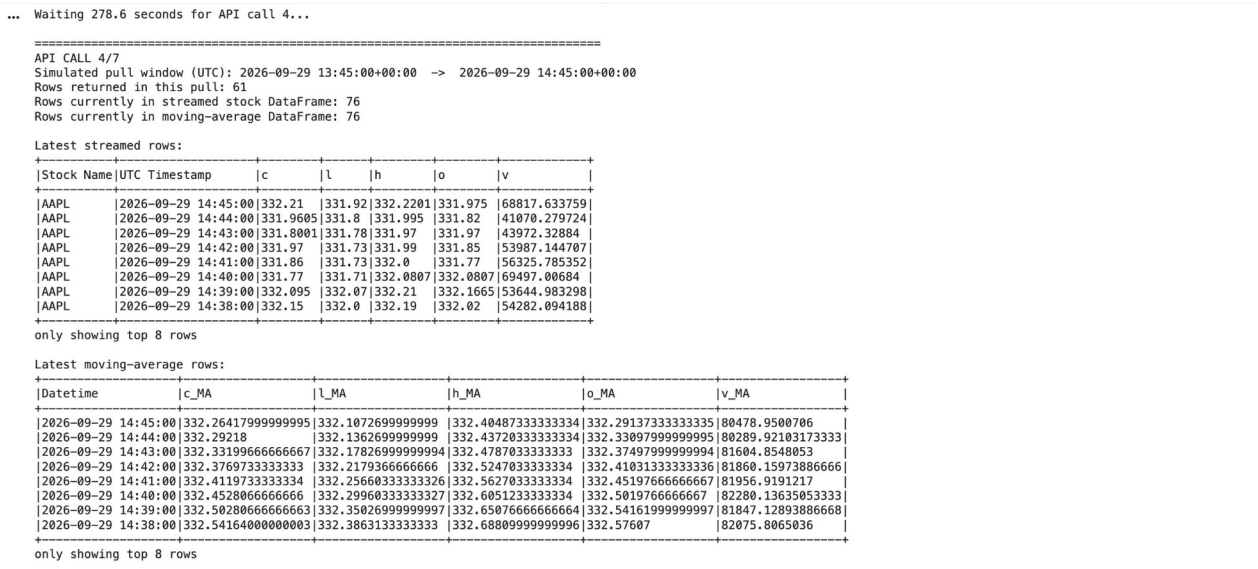
================================================================================
API CALL 1/7
Simulated pull window (UTC): 2026-09-29 13:30:00+00:00  ->  2026-09-29 14:30:00+00:00
Rows returned in this pull: 61
Rows currently in streamed stock DataFrame: 61
Rows currently in moving-average DataFrame: 61

Latest streamed rows:
+----------+--------------------+-------+------+--------+-------+-------------+
|Stock Name|UTC Timestamp       |c      |l     |h       |o      |v            |
+----------+--------------------+-------+------+--------+-------+-------------+
|AAPL      |2026-09-29 14:30:00|332.14 |331.9 |332.14  |332.065|84934.166105 |
|AAPL      |2026-09-29 14:29:00|332.07 |331.91|332.1599|331.94 |84229.186554 |
|AAPL      |2026-09-29 14:28:00|331.94 |331.63|332.0555|332.04 |150985.502036|
|AAPL      |2026-09-29 14:27:00|332.045|332.0 |332.25  |332.165|111818.300375|
|AAPL      |2026-09-29 14:26:00|332.17 |332.07|332.31  |332.3  |94004.080034 |
|AAPL      |2026-09-29 14:25:00|332.315|332.2 |332.37  |332.305|86855.887722 |
|AAPL      |2026-09-29 14:24:00|332.305|332.3 |332.58  |332.55 |68710.797874 |
|AAPL      |2026-09-29 14:23:00|332.555|332.4 |332.88  |332.78 |147709.193672|
+----------+--------------------+-------+------+--------+-------+-------------+
only showing top 8 rows

Latest moving-average rows:
+-------------------+------------------+------------------+------------------+------------------+------------------+
|Datetime           |c_MA              |l_MA              |h_MA              |o_MA              |v_MA              |
+-------------------+------------------+------------------+------------------+------------------+------------------+
|2026-09-29 14:30:00|332.80481000000003|332.65785333333326|332.9296          |332.8354466666667 |81012.94048783333|
|2026-09-29 14:29:00|332.8336433333334 |332.68552         |332.9649333333334 |332.86128         |80770.20844650002|
|2026-09-29 14:28:00|332.85964333333334|332.7131866666667 |332.99027         |332.89328         |80197.21232853335|
|2026-09-29 14:27:00|332.89231         |332.75052         |333.03008666666665|332.9324466666667 |78028.2695972    |
|2026-09-29 14:26:00|332.9314766666667 |332.77885333333336|333.06308666666666|332.95561333333336|77219.07281180001|
|2026-09-29 14:25:00|332.9554733333334 |332.80352000000005|333.0930866666667 |332.9859466666667 |76541.42176283334|
|2026-09-29 14:24:00|332.98614000000003|332.83152         |333.12742         |333.02111333333335|77798.32105943332|
|2026-09-29 14:23:00|333.02214000000004|332.86402000000004|333.1580866666666 |333.04678         |77764.59671616666|
+-------------------+------------------+------------------+------------------+------------------+------------------+
only showing top 8 rows
```

Figure 5. API Call 1/7: first one-hour pull, producing 61 cumulative rows.

By API Call 4/7, the simulated window had moved forward by 15 minutes to 13:45-14:45 UTC. The cumulative DataFrame contained 76 rows. This confirms that overlapping records were being replaced rather than duplicated; each additional pull contributed only the newly exposed five-minute portion.



```
Waiting 278.6 seconds for API call 4...

================================================================================
API CALL 4/7
Simulated pull window (UTC): 2026-09-29 13:45:00+00:00  ->  2026-09-29 14:45:00+00:00
Rows returned in this pull: 61
Rows currently in streamed stock DataFrame: 76
Rows currently in moving-average DataFrame: 76

Latest streamed rows:
+----------+--------------------+--------+------+--------+--------+-------------+
|Stock Name|UTC Timestamp       |c       |l     |h       |o       |v            |
+----------+--------------------+--------+------+--------+--------+-------------+
|AAPL      |2026-09-29 14:45:00|332.21  |331.92|332.2201|331.975 |68817.633759|
|AAPL      |2026-09-29 14:44:00|331.9605|331.8 |331.995 |331.82  |41070.279724|
|AAPL      |2026-09-29 14:43:00|331.8001|331.78|331.97  |331.97  |43972.32884 |
|AAPL      |2026-09-29 14:42:00|331.97  |331.73|331.99  |331.85  |53987.144707|
|AAPL      |2026-09-29 14:41:00|331.86  |331.73|332.0   |331.77  |56325.785352|
|AAPL      |2026-09-29 14:40:00|331.77  |331.71|332.0807|332.0807|69497.00684 |
|AAPL      |2026-09-29 14:39:00|332.095 |332.07|332.21  |332.1665|53644.983298|
|AAPL      |2026-09-29 14:38:00|332.15  |332.0 |332.19  |332.02  |54282.094188|
+----------+--------------------+--------+------+--------+--------+-------------+
only showing top 8 rows

Latest moving-average rows:
+-------------------+------------------+------------------+------------------+------------------+------------------+
|Datetime           |c_MA              |l_MA              |h_MA              |o_MA              |v_MA              |
+-------------------+------------------+------------------+------------------+------------------+------------------+
|2026-09-29 14:45:00|332.26417999999995|332.1072699999999 |332.40487333333334|332.29137333333335|80478.9500706    |
|2026-09-29 14:44:00|332.29218         |332.1362699999999 |332.43720333333334|332.33097999999995|80289.92103173333|
|2026-09-29 14:43:00|332.33199666666667|332.17826999999994|332.4787033333333 |332.37497999999994|81604.8548053    |
|2026-09-29 14:42:00|332.3769733333333 |332.2179366666666 |332.5247033333334 |332.41031333333336|81860.15973886666|
|2026-09-29 14:41:00|332.4119733333334 |332.25660333333326|332.5627033333334 |332.45197666666667|81956.9191217    |
|2026-09-29 14:40:00|332.4528066666666 |332.29960333333327|332.6051233333334 |332.5019766666667 |82280.13635053333|
|2026-09-29 14:39:00|332.50280666666663|332.35026999999997|332.65076666666664|332.54161999999997|81847.12893886668|
|2026-09-29 14:38:00|332.54164000000003|332.3863133333333 |332.68809999999996|332.57607         |82075.8065036    |
+-------------------+------------------+------------------+------------------+------------------+------------------+
only showing top 8 rows
```

Figure 6. API Call 4/7: the cumulative DataFrames have grown to 76 rows.

The seventh and final call used the window 14:00-15:00 UTC. The cumulative streamed DataFrame and moving-average DataFrame both reached 91 rows. The full retained time range is therefore 13:30-15:00 UTC, which is approximately 1.5 hours as required.

```
Waiting 265.1 seconds for API call 7...

================================================================================
API CALL 7/7
Simulated pull window (UTC): 2026-09-29 14:00:00+00:00  ->  2026-09-29 15:00:00+00:00
Rows returned in this pull: 61
Rows currently in streamed stock DataFrame: 91
Rows currently in moving-average DataFrame: 91

Latest streamed rows:
+----------+--------------------+-------+------+-------+-------+------------+
|Stock Name|UTC Timestamp       |c      |l     |h      |o      |v           |
+----------+--------------------+-------+------+-------+-------+------------+
|AAPL      |2026-09-29 15:00:00|332.46 |332.25|332.465|332.43 |63097.388633|
|AAPL      |2026-09-29 14:59:00|332.42 |332.4 |332.62 |332.61 |49698.028786|
|AAPL      |2026-09-29 14:58:00|332.61 |332.56|332.74 |332.72 |67378.162079|
|AAPL      |2026-09-29 14:57:00|332.73 |332.66|332.78 |332.68 |39778.61133 |
|AAPL      |2026-09-29 14:56:00|332.66 |332.54|332.72 |332.685|51803.61089 |
|AAPL      |2026-09-29 14:55:00|332.685|332.57|332.73 |332.585|37526.752825|
|AAPL      |2026-09-29 14:54:00|332.585|332.52|332.75 |332.63 |43252.583759|
|AAPL      |2026-09-29 14:53:00|332.65 |332.41|332.71 |332.535|44351.716636|
+----------+--------------------+-------+------+-------+-------+------------+
only showing top 8 rows

Latest moving-average rows:
+--------------------+-----------------+-----------------+-----------------+-----------------+-----------------+
|Datetime            |c_MA             |l_MA             |h_MA             |o_MA             |v_MA             |
+--------------------+-----------------+-----------------+-----------------+-----------------+-----------------+
|2026-09-29 15:00:00|332.22838999999993|332.0766233333333 |332.34906000000007|332.2199366666667 |64617.42990330001|
|2026-09-29 14:59:00|332.2177233333333 |332.06495666666666|332.33822666666674|332.20777         |65345.32248570002|
|2026-09-29 14:58:00|332.20605666666665|332.04862333333335|332.32289         |332.18543666666665|66496.36107796668|
|2026-09-29 14:57:00|332.18372333333326|332.01762333333335|332.3000733333333 |332.16276999999997|69283.2724098667 |
|2026-09-29 14:56:00|332.16089         |331.99562333333336|332.2824066666666 |332.1456033333333 |71684.59537803337|
|2026-09-29 14:55:00|332.1445566666666 |331.9799566666667 |332.2687399999999 |332.13277000000005|73091.27768283336|
|2026-09-29 14:54:00|332.13222333333334|331.96762333333334|332.25674         |332.1234366666667 |74735.58217940004|
|2026-09-29 14:53:00|332.12289000000004|331.96029000000004|332.2510733333333 |332.12077000000005|75584.1893165667 |
+--------------------+-----------------+-----------------+-----------------+-----------------+-----------------+
only showing top 8 rows
```

Figure 7. API Call 7/7: final window and 91 cumulative rows.

The row-count progression was 61, 66, 71, 76, 81, 86, and 91. Because every new one-hour pull starts five minutes later than the previous one, most rows overlap. The incremental update correctly preserves the newest version of overlapping timestamps and adds roughly five new rows per call.

4. Final Streamed Stock DataFrame

The final streamed DataFrame follows the required schema: Stock Name, UTC Timestamp, c, l, h, o, and v. The first retained timestamp is 13:30 UTC. The screenshot below shows the beginning of the final DataFrame.

```
################################################################################
FINAL STREAMED STOCK DATAFRAME
################################################################################
+----------+-------------------+--------+--------+--------+--------+-------------+
|Stock Name|UTC Timestamp      |c       |l       |h       |o       |v            |
+----------+-------------------+--------+--------+--------+--------+-------------+
|AAPL      |2026-09-29 13:30:00|333.2079|333.2079|337.085 |336.965 |778633.038247|
|AAPL      |2026-09-29 13:31:00|332.345 |332.06  |333.89  |333.2   |297199.164806|
|AAPL      |2026-09-29 13:32:00|333.085 |332.19  |333.38  |332.29  |235648.406543|
|AAPL      |2026-09-29 13:33:00|333.11  |332.835 |333.96  |333.08  |185590.319469|
|AAPL      |2026-09-29 13:34:00|333.71  |332.87  |333.86  |333.1456|145387.851578|
|AAPL      |2026-09-29 13:35:00|334.15  |333.43  |334.18  |333.7431|162887.797191|
|AAPL      |2026-09-29 13:36:00|333.875 |333.66  |334.3999|334.145 |124533.621552|
|AAPL      |2026-09-29 13:37:00|334.05  |333.86  |334.54  |333.87  |100647.332178|
|AAPL      |2026-09-29 13:38:00|333.79  |333.3   |334.09  |334.075 |118314.471034|
|AAPL      |2026-09-29 13:39:00|333.74  |333.35  |333.98  |333.795 |106288.568204|
|AAPL      |2026-09-29 13:40:00|333.5   |332.9   |333.76  |333.7352|284160.455215|
|AAPL      |2026-09-29 13:41:00|333.05  |333.045 |333.73  |333.5   |226869.731994|
|AAPL      |2026-09-29 13:42:00|332.925 |332.78  |333.1   |333.08  |138080.204217|
|AAPL      |2026-09-29 13:43:00|332.31  |332.26  |332.98  |332.93  |102135.076914|
|AAPL      |2026-09-29 13:44:00|332.525 |332.25  |332.75  |332.35  |128495.195291|
|AAPL      |2026-09-29 13:45:00|332.915 |332.5   |333.05  |332.5   |116709.559247|
|AAPL      |2026-09-29 13:46:00|333.385 |332.77  |333.6299|332.92  |203601.017836|
|AAPL      |2026-09-29 13:47:00|333.09  |333.04  |333.5   |333.38  |79183.634692 |
|AAPL      |2026-09-29 13:48:00|332.878 |332.66  |333.17  |333.085 |119682.732746|
|AAPL      |2026-09-29 13:49:00|332.945 |332.65  |332.975 |332.8851|103871.741078|
|AAPL      |2026-09-29 13:50:00|332.7701|332.66  |333.0656|332.925 |95202.395655 |
|AAPL      |2026-09-29 13:51:00|332.945 |332.72  |333.3   |332.795 |96317.85849  |
|AAPL      |2026-09-29 13:52:00|333.075 |332.9246|333.3   |332.945 |90907.475946 |
|AAPL      |2026-09-29 13:53:00|333.3201|333.07  |333.41  |333.07  |88588.36497  |
|AAPL      |2026-09-29 13:54:00|333.385 |333.275 |333.5   |333.32  |67699.067576 |
|AAPL      |2026-09-29 13:55:00|333.235 |333.04  |333.4   |333.36  |124562.86662 |
|AAPL      |2026-09-29 13:56:00|332.8899|332.81  |333.21  |333.21  |73674.548565 |
|AAPL      |2026-09-29 13:57:00|333.22  |332.85  |333.24  |332.86  |87542.396813 |
|AAPL      |2026-09-29 13:58:00|332.92  |332.75  |333.25  |333.215 |85917.220096 |
|AAPL      |2026-09-29 13:59:00|332.85  |332.74  |332.92  |332.9   |67039.303015 |
|AAPL      |2026-09-29 14:00:00|333.005 |332.73  |333.2   |332.84  |77652.204865 |
|AAPL      |2026-09-29 14:01:00|333.09  |332.79  |333.12  |333.005 |70495.765569 |
|AAPL      |2026-09-29 14:02:00|333.04  |332.76  |333.14  |333.09  |289252.540292|
```

Figure 8. Beginning of the final streamed AAPL DataFrame at 13:30 UTC.

The end of the DataFrame reaches 15:00 UTC, confirming that the final cumulative stream spans about 90 minutes. The last rows are shown below.

```
|AAPL      |2026-09-29 14:41:00|331.86  |331.73  |332.0   |331.77  |56325.785352 |
|AAPL      |2026-09-29 14:42:00|331.97  |331.73  |331.99  |331.85  |53987.144707 |
|AAPL      |2026-09-29 14:43:00|331.8001|331.78  |331.97  |331.97  |43972.32884  |
|AAPL      |2026-09-29 14:44:00|331.9605|331.8   |331.995 |331.82  |41070.279724 |
|AAPL      |2026-09-29 14:45:00|332.21  |331.92  |332.2201|331.975 |68817.633759 |
|AAPL      |2026-09-29 14:46:00|332.16  |332.11  |332.315 |332.23  |56944.409648 |
|AAPL      |2026-09-29 14:47:00|332.07  |332.01  |332.165 |332.14  |45116.633142 |
|AAPL      |2026-09-29 14:48:00|332.02  |332.0   |332.145 |332.09  |48495.461633 |
|AAPL      |2026-09-29 14:49:00|331.94  |331.85  |332.071 |332.04  |49340.930353 |
|AAPL      |2026-09-29 14:50:00|332.27  |331.94  |332.27  |331.94  |57586.935006 |
|AAPL      |2026-09-29 14:51:00|332.465 |332.18  |332.74  |332.265 |94720.621211 |
|AAPL      |2026-09-29 14:52:00|332.5112|332.4006|332.59  |332.4761|46712.900595 |
|AAPL      |2026-09-29 14:53:00|332.65  |332.41  |332.71  |332.535 |44351.716636 |
|AAPL      |2026-09-29 14:54:00|332.585 |332.52  |332.75  |332.63  |43252.583759 |
|AAPL      |2026-09-29 14:55:00|332.685 |332.57  |332.73  |332.585 |37526.752825 |
|AAPL      |2026-09-29 14:56:00|332.66  |332.54  |332.72  |332.685 |51803.61089  |
|AAPL      |2026-09-29 14:57:00|332.73  |332.66  |332.78  |332.68  |39778.61133  |
|AAPL      |2026-09-29 14:58:00|332.61  |332.56  |332.74  |332.72  |67378.162079 |
|AAPL      |2026-09-29 14:59:00|332.42  |332.4   |332.62  |332.61  |49698.028786 |
|AAPL      |2026-09-29 15:00:00|332.46  |332.25  |332.465 |332.43  |63097.388633 |
+----------+-------------------+--------+--------+--------+--------+-------------+
```

Figure 9. End of the final streamed DataFrame at 15:00 UTC.

5. Final 30-Minute Moving-Average DataFrame

The separate moving-average DataFrame contains Datetime together with c_MA, l_MA, h_MA, o_MA, and v_MA. The screenshot below shows the first part of the final result, beginning at 13:30 UTC.

```
################################################################################
FINAL 30-MINUTE MOVING-AVERAGE DATAFRAME
################################################################################
+------------------+------------------+------------------+------------------+------------------+------------------+
|Datetime          |c_MA              |l_MA              |h_MA              |o_MA              |v_MA              |
+------------------+------------------+------------------+------------------+------------------+------------------+
|2026-09-29 13:30:00|333.2079          |333.2079          |337.085           |336.965           |778633.038247     |
|2026-09-29 13:31:00|332.77645         |332.63395         |335.48749999999995|335.0825          |537916.1015265    |
|2026-09-29 13:32:00|332.8793          |332.4859666666667 |334.78499999999997|334.15166666666664|437160.2031986666 |
|2026-09-29 13:33:00|332.93697499999996|332.57322500000004|334.57874999999996|333.88374999999996|374267.73226624995|
|2026-09-29 13:34:00|333.09157999999996|332.6325800000001 |334.43499999999995|333.73611999999997|328491.7561286    |
|2026-09-29 13:35:00|333.26798333333335|332.7654833333334 |334.3925          |333.7372833333333 |300891.09630566667|
|2026-09-29 13:36:00|333.3547          |332.8932714285715 |334.3935571428571 |333.7955285714285 |275697.1713408571 |
|2026-09-29 13:37:00|333.4416125       |333.01411250000007|334.4118625       |333.80483749999996|253815.94144549998|
|2026-09-29 13:38:00|333.48032222222224|333.04587777777783|334.3761          |333.8348555555555 |238760.22251088885|
|2026-09-29 13:39:00|333.50629         |333.0762900000001 |334.33649         |333.83086999999995|225513.0570802    |
|2026-09-29 13:40:00|333.50571818181817|333.0602636363637 |334.2840818181818 |333.8221727272727 |230844.6387288182 |
|2026-09-29 13:41:00|333.46774166666665|333.0589916666667 |334.23790833333334|333.795325        |230513.39650091666|
|2026-09-29 13:42:00|333.42599230769235|333.03753076923084|334.1503769230769 |333.7403          |223403.15094061536|
|2026-09-29 13:43:00|333.3462785714286 |332.9819928571429 |334.0667785714286 |333.68242142857144|214741.14565299996|
|2026-09-29 13:44:00|333.2915266666667 |332.9331933333334 |333.97899333333334|333.59359333333333|208991.41562886664|
|2026-09-29 13:45:00|333.26799375      |332.9061187500001 |333.92093125      |333.52524375      |203223.79960499998|
|2026-09-29 13:46:00|333.27487647058825|332.8981117647059 |333.9038117647059 |333.4896411764706 |203245.98891270586|
|2026-09-29 13:47:00|333.2646055555556 |332.9059444444445 |333.8813777777778 |333.48355000000004|196353.63590044444|
|2026-09-29 13:48:00|333.24425789473685|332.8930473684211 |333.8439368421053 |333.46257368421055|192318.32520810523|
|2026-09-29 13:49:00|333.229295        |332.880895        |333.80049         |333.43370000000004|187895.9960016    |
|2026-09-29 13:50:00|333.2074285714286 |332.8703761904762 |333.76549523809524|333.40947619047625|183482.0150327143 |
|2026-09-29 13:51:00|333.1955          |332.86354090909094|333.7443363636364 |333.3815454545455 |179520.00791713636|
|2026-09-29 13:52:00|333.1902608695652 |332.86619565217393|333.7250173913044 |333.36256521739136|175667.2891357826 |
|2026-09-29 13:53:00|333.1956708333333 |332.8746875       |333.7118916666667 |333.35037500000004|172039.000628875  |
|2026-09-29 13:54:00|333.203244        |332.8907          |333.703416        |333.34916000000004|167865.40330675998|
|2026-09-29 13:55:00|333.2044653846154 |332.8964423076923 |333.6917461538461 |333.349576923077  |166199.92112649998|
|2026-09-29 13:56:00|333.1928148148148 |332.89324074074074|333.6739037037037 |333.34440740740746|162773.05547607408|
|2026-09-29 13:57:00|333.1937857142857 |332.89169642857144|333.6584071428571 |333.3271071428572 |160086.24623810712|
|2026-09-29 13:58:00|333.1843448275862 |332.8868103448276 |333.644324137931  |333.3232413793104 |157528.6936125172 |
|2026-09-29 13:59:00|333.1732          |332.88191666666665|333.62017999999995|333.30913333333336|154512.38059259998|
|2026-09-29 14:00:00|333.16643666666664|332.8659866666667 |333.49068000000005|333.1716333333333 |131146.3528132    |
|2026-09-29 14:01:00|333.19127         |332.89032000000003|333.4650133333334 |333.16513333333324|123589.57283863335|
|2026-09-29 14:02:00|333.18977         |332.90932000000004|333.4570133333334 |333.19179999999994|125376.37729693335|
```

Figure 10. Beginning of the final 30-minute moving-average DataFrame.

The moving-average output continues through 15:00 UTC. Because the calculation uses a trailing time window, early timestamps use the data available so far, while later timestamps use the full 30-minute window whenever those observations are present.

```
|2026-09-29 14:41:00|332.4119733333334 |332.25660333333326|332.5627033333334 |332.45197666666667|81956.9191217     |
|2026-09-29 14:42:00|332.3769733333333 |332.2179366666666 |332.5247033333334 |332.41031333333336|81860.15973886666 |
…|2026-09-29 14:43:00|332.33199666666667|332.17826999999994|332.4787033333333 |332.37497999999994|81604.8548053     |
|2026-09-29 14:44:00|332.29218         |332.1362699999999 |332.43720333333334|332.33097999999995|80289.92103173333 |
|2026-09-29 14:45:00|332.26417999999995|332.1072699999999 |332.40487333333334|332.29137333333335|80478.9500706     |
|2026-09-29 14:46:00|332.2325133333333 |332.08427         |332.37770666666677|332.2633733333333 |80029.11541663333 |
|2026-09-29 14:47:00|332.2025133333333 |332.05227         |332.34454000000005|332.23170999999996|80056.52805093332 |
|2026-09-29 14:48:00|332.1755133333333 |332.0256033333333 |332.3140400000001 |332.2032333333333 |79954.9585784     |
|2026-09-29 14:49:00|332.1411833333333 |331.9949366666667 |332.2840733333334 |332.1769000000001 |79670.38518466665 |
|2026-09-29 14:50:00|332.13251666666673|331.97527         |332.25907333333345|332.14256666666665|79019.47033993334 |
|2026-09-29 14:51:00|332.12818333333337|331.9656033333334 |332.26274000000006|332.13340000000005|79120.58540303333 |
|2026-09-29 14:52:00|332.1197233333334 |331.95995666666664|332.25674000000004|332.12893666666673|79029.4385511     |
|2026-09-29 14:53:00|332.12289000000004|331.96029000000004|332.2510733333333 |332.12077000000005|75584.1893165667  |
|2026-09-29 14:54:00|332.13222333333334|331.96762333333334|332.25674         |332.1234366666667 |74735.58217940004 |
|2026-09-29 14:55:00|332.1445566666666 |331.9799566666667 |332.2687399999999 |332.13277000000005|73091.27768283336 |
|2026-09-29 14:56:00|332.16089         |331.99562333333336|332.2824066666666 |332.1456033333333 |71684.59537803337 |
|2026-09-29 14:57:00|332.18372333333326|332.01762333333335|332.3000733333333 |332.16276999999997|69283.2724098667  |
|2026-09-29 14:58:00|332.20605666666665|332.04862333333335|332.32289         |332.18543666666665|66496.36107796668 |
|2026-09-29 14:59:00|332.2177233333333 |332.06495666666666|332.33822666666674|332.20777         |65345.32248570002 |
|2026-09-29 15:00:00|332.22838999999993|332.0766233333333 |332.34906000000007|332.2199366666667 |64617.42990330001 |
+-------------------+------------------+------------------+------------------+------------------+------------------+
```

Figure 11. End of the final moving-average DataFrame at 15:00 UTC.

Final validation

The final schema output confirms that the streamed DataFrame and moving-average DataFrame use the expected column names and numeric types. The notebook also reports that seven API calls were attempted and ends with Done, confirming that the full simulation completed.

```
Final stock schema:
root
 |-- Stock Name: string (nullable = false)
 |-- UTC Timestamp: timestamp (nullable = false)
 |-- c: double (nullable = true)
 |-- l: double (nullable = true)
 |-- h: double (nullable = true)
 |-- o: double (nullable = true)
 |-- v: double (nullable = true)


Final moving-average schema:
root
 |-- Datetime: timestamp (nullable = false)
 |-- c_MA: double (nullable = true)
 |-- l_MA: double (nullable = true)
 |-- h_MA: double (nullable = true)
 |-- o_MA: double (nullable = true)
 |-- v_MA: double (nullable = true)


Total API calls attempted: 7
Done.
```

Figure 12. Final schemas and confirmation that all seven API calls were completed.

6. Section 1 Summary

This implementation simulates streaming stock analytics with repeated historical API pulls. Seven one-hour AAPL requests were made five minutes apart. A Spark DataFrame was maintained incrementally by replacing overlapping timestamps with the latest pull, which produced 91 cumulative rows spanning approximately 1.5 hours. After every update, a separate Spark DataFrame stored trailing 30-minute moving averages for close, low, high, open, and volume. The final results show that both the streamed data and the moving-average data were updated successfully through the last simulated timestamp.

Section 2: Airflow Data Pipelining

EECS E6893 Big Data Analytics - Homework 2

Fall 2026

Yihang Sun

This report documents the Airflow setup, the HelloWorld workflow, the required 19-task workflow, and the end-to-end stock watchlist pipeline.

Task 1 - HelloWorld

Q1.1 Airflow Installation and Startup

I installed Apache Airflow locally in a Python virtual environment and configured it to use SequentialExecutor for this assignment. I started the webserver on port 8081 and ran the scheduler in a separate terminal. The terminal outputs below show that both services started successfully.

```
(e6893_hw2_airflow_venv) (base) yihangsun@Kevins-MacBook-Pro-2 ~ % airflow webserver --port 8081
  ____________       _____________
 ____    |__( )_________  __/__  /________      __
____  /| |_  /__  ___/_  /_ __  /_  __ \_ | /| / /
___  ___ |  / _  /    _  __/ _  / / /_/ /_ |/ |/ /
 _/_/  |_/_/  /_/    /_/    /_/  \____/____/|__/
Running the Gunicorn Server with:
Workers: 4 sync
Host: 0.0.0.0:8081
Timeout: 120
Logfiles: - -
Access Logformat:
==================================================================
/Users/yihangsun/e6893_hw2_airflow_venv/lib/python3.11/site-packages/flask_limiter/extension.py:333 UserWarning: Using the in-memory storage for tracking rate limits
 as no storage was explicitly specified. This is not recommended for production use. See: https://flask-limiter.readthedocs.io#configuring-a-storage-backend for docu
mentation about configuring the storage backend.
/Users/yihangsun/e6893_hw2_airflow_venv/lib/python3.11/site-packages/airflow/api_connexion/schemas/task_schema.py:52 ChangedInMarshmallow4Warning: `Number` field sho
uld not be instantiated. Use `Integer`, `Float`, or `Decimal` instead.
/Users/yihangsun/e6893_hw2_airflow_venv/lib/python3.11/site-packages/airflow/api_connexion/schemas/task_schema.py:55 ChangedInMarshmallow4Warning: `Number` field sho
uld not be instantiated. Use `Integer`, `Float`, or `Decimal` instead.
/Users/yihangsun/e6893_hw2_airflow_venv/lib/python3.11/site-packages/airflow/api_connexion/schemas/task_schema.py:59 ChangedInMarshmallow4Warning: `Number` field sho
uld not be instantiated. Use `Integer`, `Float`, or `Decimal` instead.
[2026-10-06 19:43:20 -0400] [15796] [INFO] Starting gunicorn 23.0.0
[2026-10-06 19:43:20 -0400] [15796] [INFO] Listening at: http://0.0.0.0:8081 (15796)
[2026-10-06 19:43:20 -0400] [15796] [INFO] Using worker: sync
[2026-10-06 19:43:20 -0400] [15805] [INFO] Booting worker with pid: 15805
[2026-10-06 19:43:20 -0400] [15806] [INFO] Booting worker with pid: 15806
[2026-10-06 19:43:21 -0400] [15807] [INFO] Booting worker with pid: 15807
[2026-10-06 19:43:21 -0400] [15808] [INFO] Booting worker with pid: 15808
```

Figure 1. Airflow webserver running on port 8081.

```
(base) yihangsun@Kevins-MacBook-Pro-2 ~ % source "/Users/yihangsun/e6893_hw2_airflow_venv/bin/activate"
export AIRFLOW_HOME="/Users/yihangsun/airflow"
export AIRFLOW__CORE__EXECUTOR=SequentialExecutor
export AIRFLOW__CORE__LOAD_EXAMPLES=False
airflow scheduler
  ____________       _____________
 ____    |__( )_________  __/__  /________      __
____  /| |_  /__  ___/_  /_ __  /_  __ \_ | /| / /
___  ___ |  / _  /    _  __/ _  / / /_/ /_ |/ |/ /
 _/_/  |_/_/  /_/    /_/    /_/  \____/____/|__/
[2026-10-06T19:45:29.709-0400] {executor_loader.py:258} INFO - Loaded executor: SequentialExecutor
[2026-10-06T19:45:30.145-0400] {scheduler_job_runner.py:950} INFO - Starting the scheduler
[2026-10-06T19:45:30.145-0400] {scheduler_job_runner.py:957} INFO - Processing each file at most -1 times
[2026-10-06T19:45:30.149-0400] {manager.py:174} INFO - Launched DagFileProcessorManager with pid: 16407
[2026-10-06T19:45:30.150-0400] {scheduler_job_runner.py:1949} INFO - Adopting or resetting orphaned tasks for active dag runs
[2026-10-06 19:45:30 -0400] [16406] [INFO] Starting gunicorn 23.0.0
[2026-10-06 19:45:30 -0400] [16406] [INFO] Listening at: http://[::]:8793 (16406)
[2026-10-06 19:45:30 -0400] [16406] [INFO] Using worker: sync
[2026-10-06T19:45:30.684-0400] {settings.py:63} INFO - Configured default timezone UTC
[2026-10-06 19:45:30 -0400] [16410] [INFO] Booting worker with pid: 16410
[2026-10-06T19:45:30.692-0400] {manager.py:406} WARNING - Because we cannot use more than 1 thread (parsing_processes = 2) when using sqlite. So we set parallelism t
o 1.
[2026-10-06 19:45:30 -0400] [16411] [INFO] Booting worker with pid: 16411
[2026-10-06 19:45:34 -0400] [16406] [INFO] Handling signal: winch
[2026-10-06 19:45:34 -0400] [16406] [INFO] Handling signal: winch
```

Figure 2. Airflow scheduler started with SequentialExecutor.

After the webserver and scheduler were running, I opened the Airflow web interface in the browser. The DAG page loaded successfully and showed the DAGs used for the homework.

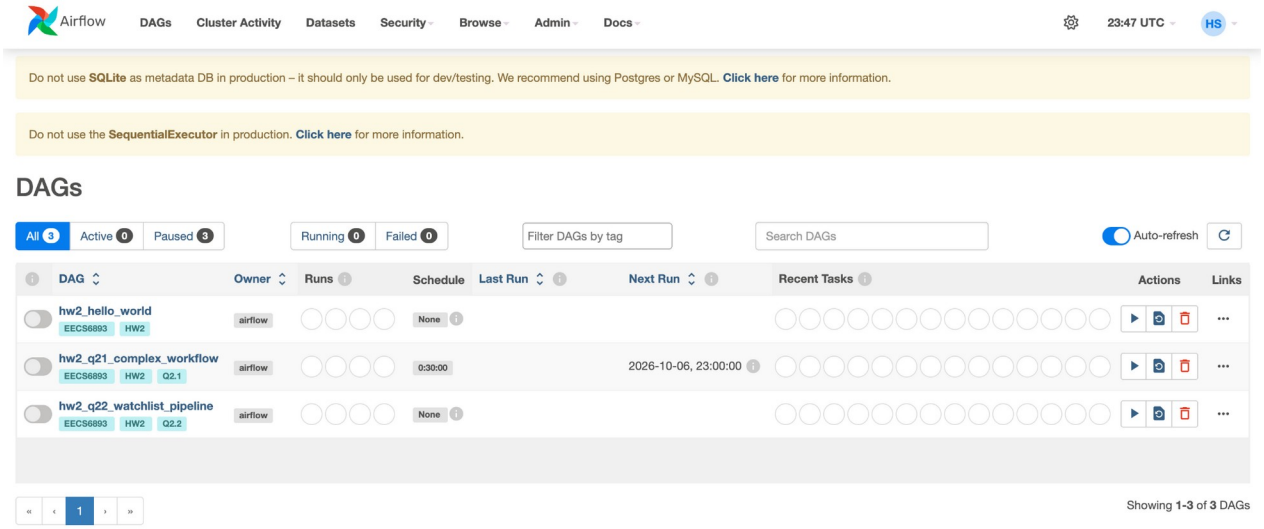


Figure 3. Airflow browser interface after successful login.

Q1.2 HelloWorld with SequentialExecutor

I ran the HelloWorld DAG with SequentialExecutor. The workflow contains three sequential tasks: hello, show_time, and finish. The Grid/Details view shows one successful DAG run and all three tasks completed successfully.

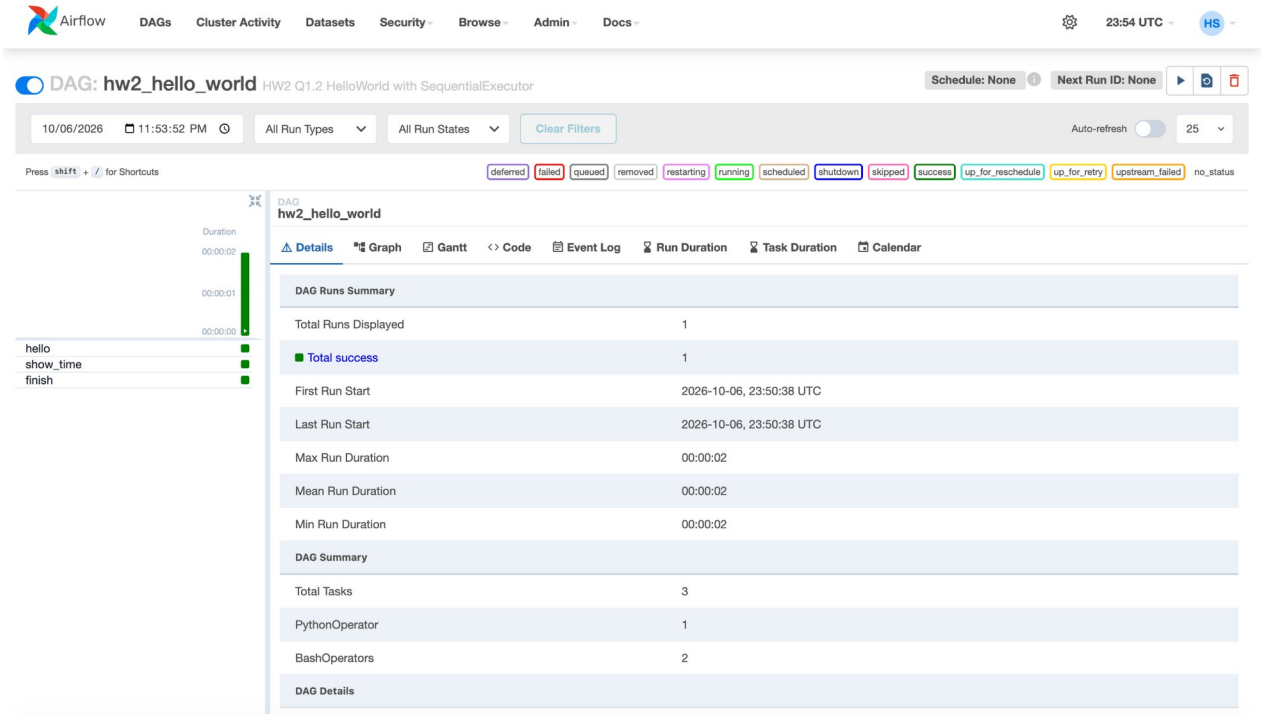


Figure 4. HelloWorld Grid/Details view showing a successful run.

The Graph view shows the dependency order of the three tasks. The tasks form a simple linear pipeline, so each task starts only after the previous task has completed.

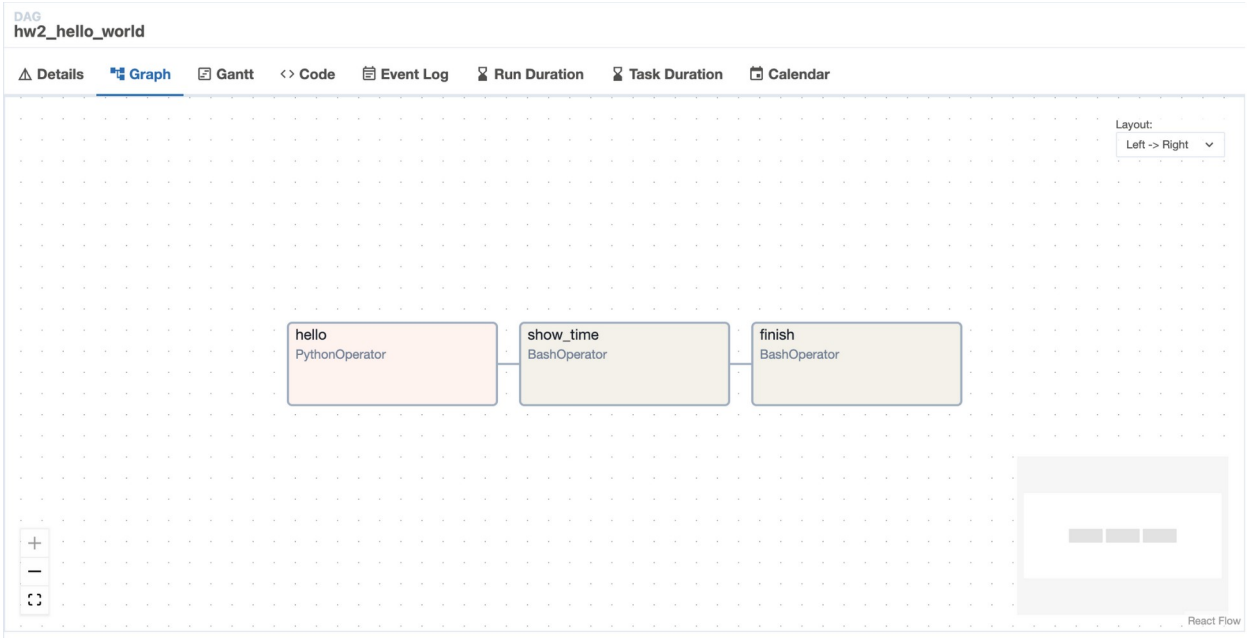


Figure 5. HelloWorld Graph view.

The Gantt view shows when each task ran during the selected DAG run. Because SequentialExecutor runs tasks one at a time, the bars appear one after another instead of overlapping.

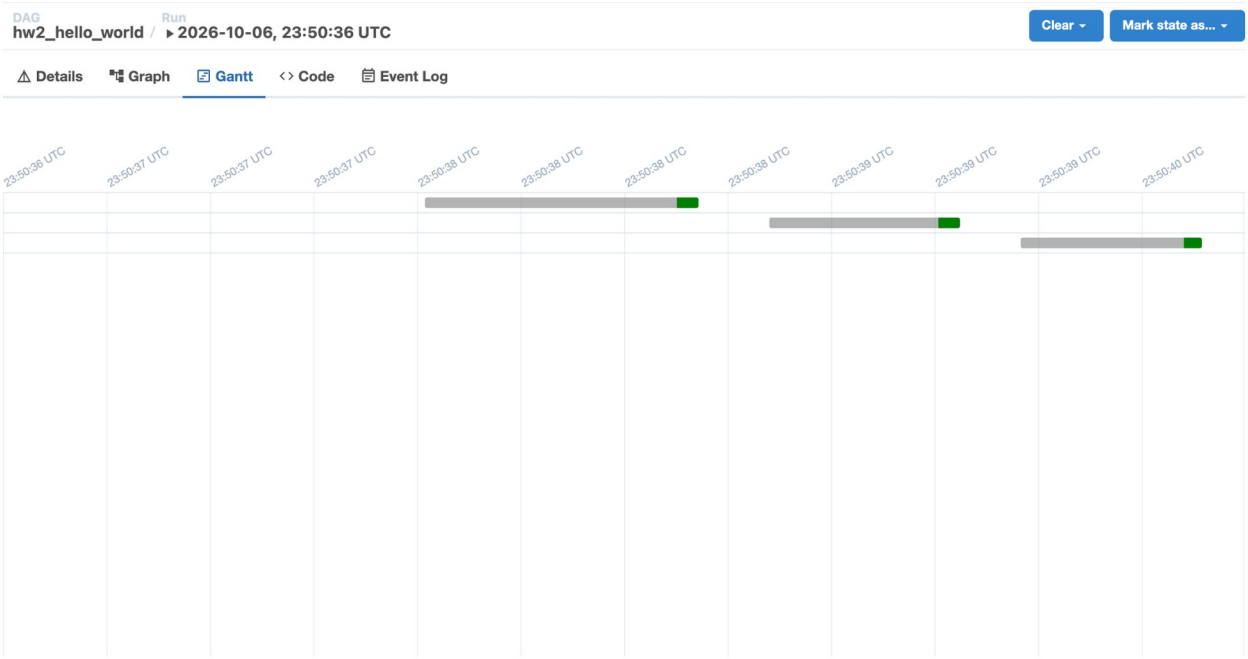


Figure 6. HelloWorld Gantt chart for the successful manual run.

Two Additional Airflow UI Features

Event Log is useful for tracking state changes and the exact time of each transition. In this example, the log records the running and success events for hello, show_time, and finish. If a task fails or stays in an unexpected state, this view helps identify which task changed state and when.

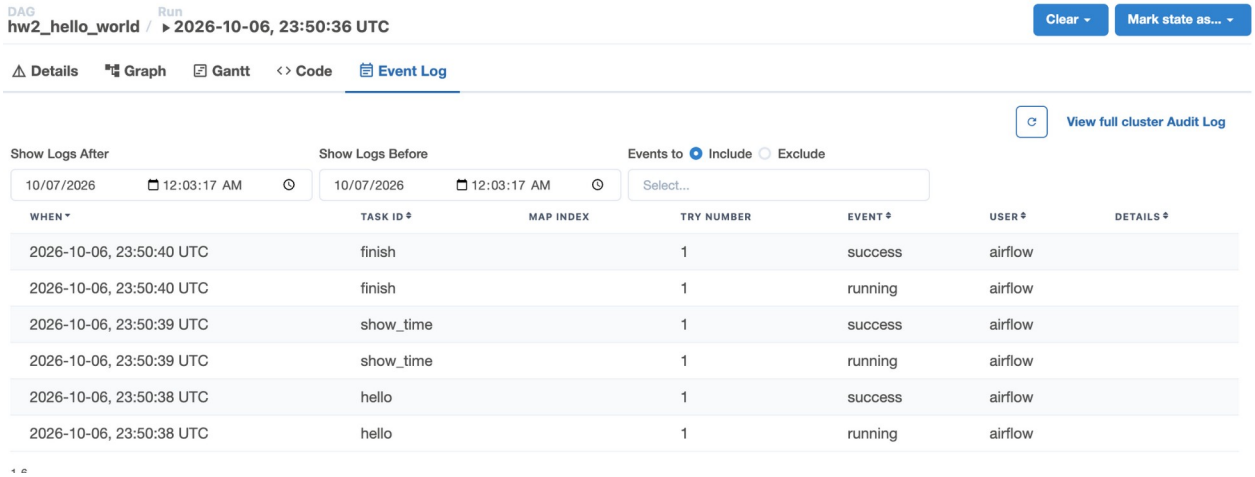


Figure 7. Event Log for the HelloWorld DAG.

Task Duration summarizes how long individual tasks take. This is useful for finding slow tasks or bottlenecks across runs. The HelloWorld tasks are very small, so their durations are close to zero, but the same visualization is much more informative for longer workflows.



Figure 8. Task Duration visualization for HelloWorld.

Task 2 - Build Workflows

Q2.1 Required 19-Task DAG

I implemented the t1-t19 workflow from the assignment/tutorial using both BashOperator and PythonOperator. The DAG contains 19 tasks in total: 11 BashOperator tasks and 8 PythonOperator tasks. The operators use different types of commands and functions, including echo/printf, date, Python launched from Bash, word counting, arithmetic, file listing, Python print, count, sleep, sum, and simple statistics.

The Grid/Details view below shows that the DAG completed successfully. All 19 task instances in the displayed runs are green.

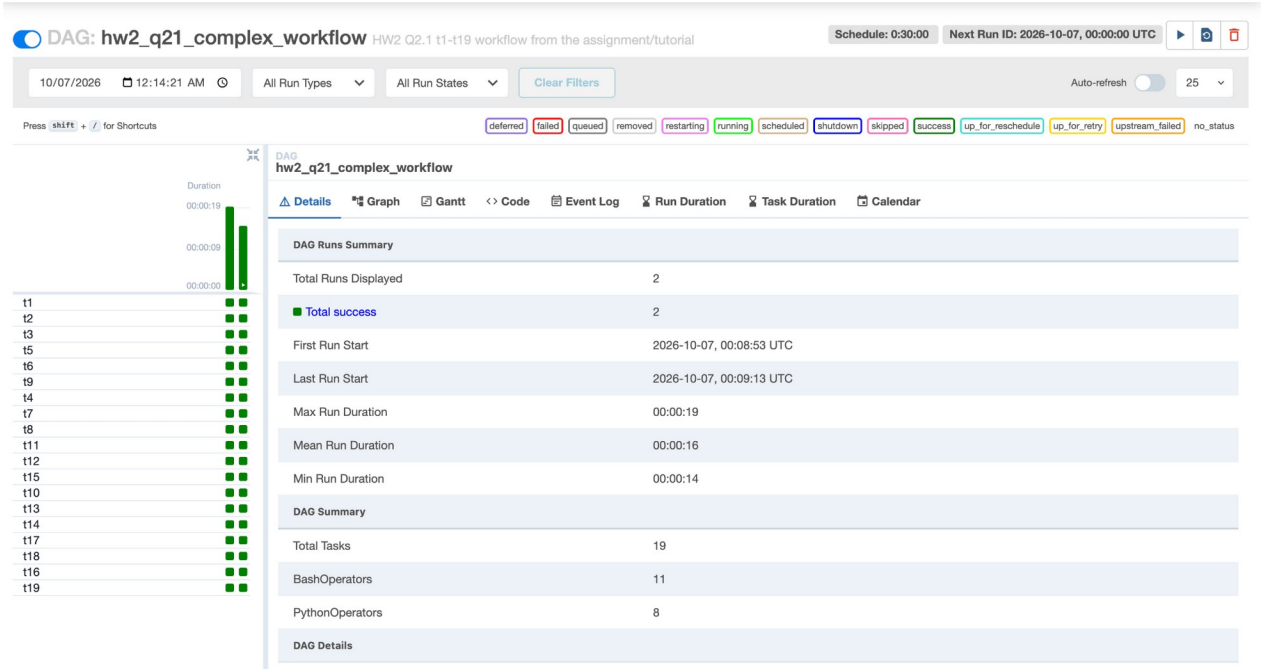


Figure 9. Q2.1 Grid/Details view with all 19 tasks successful.

The Graph view verifies that the task dependencies match the required branching and merging workflow. It also makes it easy to see where independent branches split and where they join again.

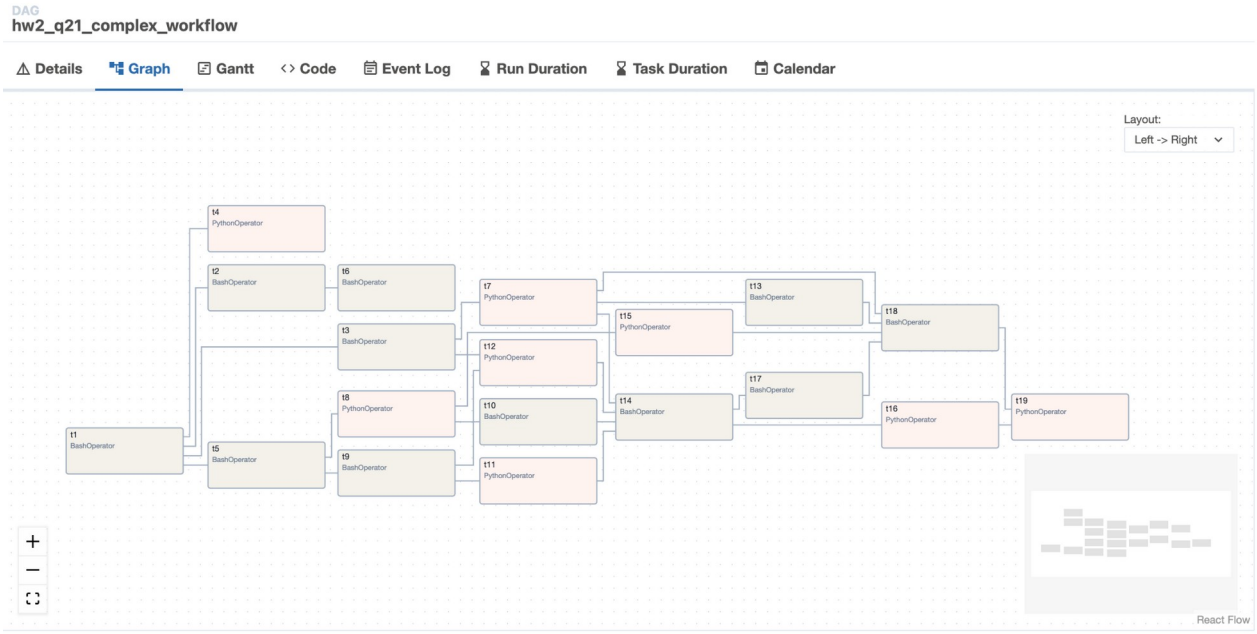


Figure 10. Q2.1 Graph view showing the full t1-t19 dependency structure.

I manually triggered the DAG and inspected the Gantt chart for that run. The chart shows the start and finish times of the individual tasks while respecting the dependency structure.

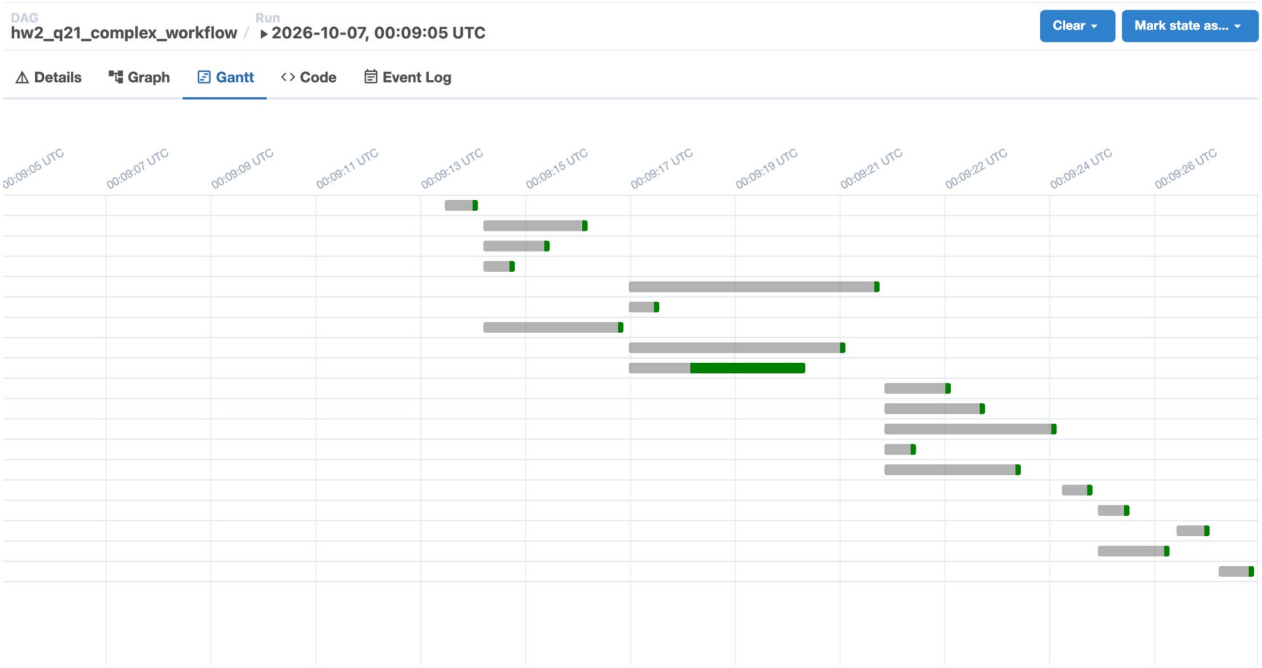


Figure 11. Q2.1 Gantt chart for a manually triggered run.

Q2.1 Scheduling Decision

I used a fixed start date of October 1, 2026 in the America/New_York timezone and set the schedule interval to 30 minutes. I also used catchup=False so Airflow would not backfill a large number of old runs from the start date. This gives the scheduler a valid past start date while allowing normal 30-minute scheduling once the DAG is enabled.

The running history below shows three successful scheduled runs with logical dates 03:30, 04:00, and 04:30 UTC. The logical dates are exactly 30 minutes apart. Each row has Run Type = scheduled and External Trigger = False, confirming that these were created by the scheduler rather than by manual triggers.

State	Dag Id	Logical Date	Run Id	Run Type	Queued At	Start Date	End Date	Note	External Trigger	Co
success	hw2_q21_complex_workflow	2026-10-07, 04:30:00	scheduled__2026-10-07T04:30:00+00:00	scheduled	2026-10-07, 05:00:01	2026-10-07, 05:00:01	2026-10-07, 05:00:20		False	{}
success	hw2_q21_complex_workflow	2026-10-07, 04:00:00	scheduled__2026-10-07T04:00:00+00:00	scheduled	2026-10-07, 04:30:01	2026-10-07, 04:30:01	2026-10-07, 04:30:18		False	{}
success	hw2_q21_complex_workflow	2026-10-07, 03:30:00	scheduled__2026-10-07T03:30:00+00:00	scheduled	2026-10-07, 04:20:43	2026-10-07, 04:20:43	2026-10-07, 04:20:59		False	{}

Figure 12. Q2.1 running history showing the first scheduled run and two 30-minute repeats.

Q2.2 End-to-End Stock Watchlist Pipeline

For the end-to-end pipeline, I selected NVDA, TSLA, and SPY as my watchlist. I used a 3.0% latest-day price-swing threshold for the volatility alert. The DAG has three sequential steps: fetch_data, analytics_summary, and volatility_alert.

The successful run below shows all three tasks in green. The fetch step downloaded the last 30 trading days for each ticker, the analytics step created the summary metrics and saved watchlist_summary.csv, and the final step checked the latest-day price swing against the 3.0% threshold.

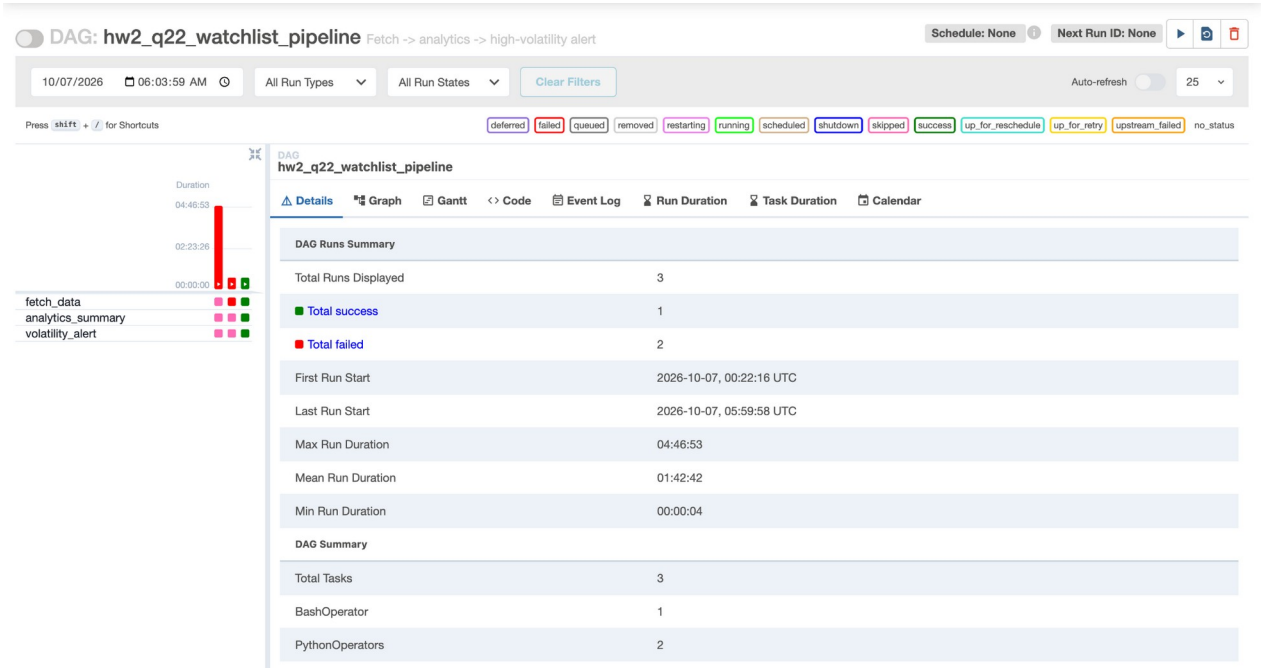


Figure 13. Q2.2 Grid/Details view showing one successful run and three successful tasks.

The Graph view confirms the required sequential order: data is fetched first, then summarized, and finally checked for a volatility alert.

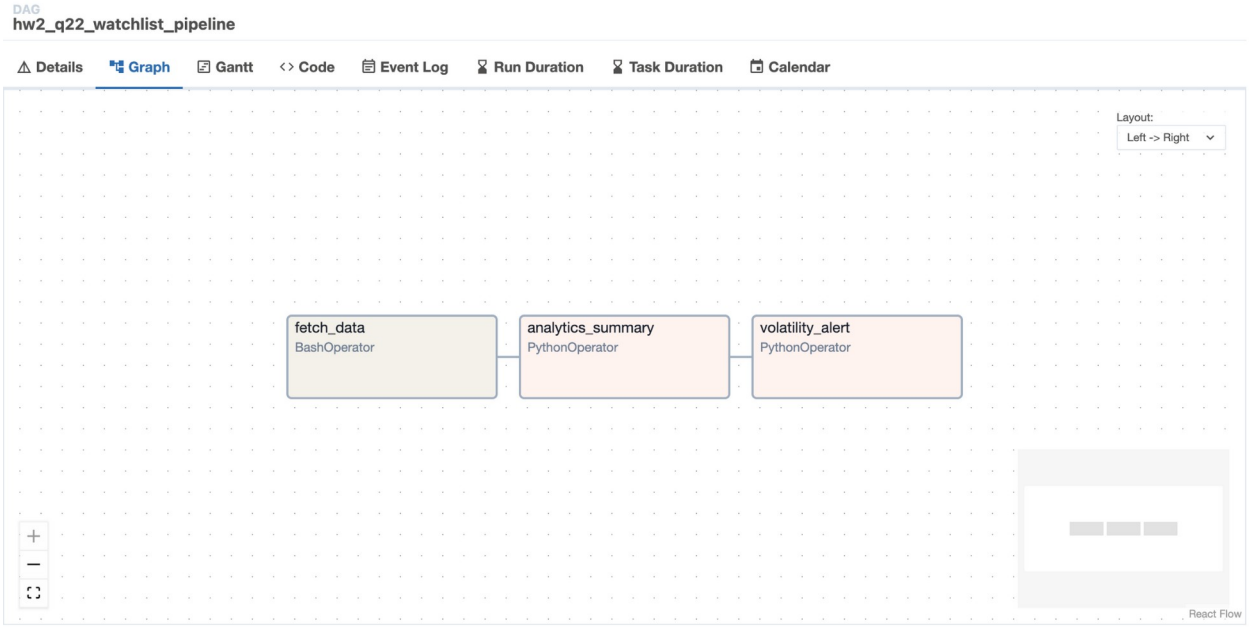


Figure 14. Q2.2 Graph view: fetch_data -> analytics_summary -> volatility_alert.

Q2.2 Task Outputs

The fetch_data log shows that 30 rows were collected for each of NVDA, TSLA, and SPY, for 90 rows in total. The combined raw data was saved successfully, and the task exited with return code 0.

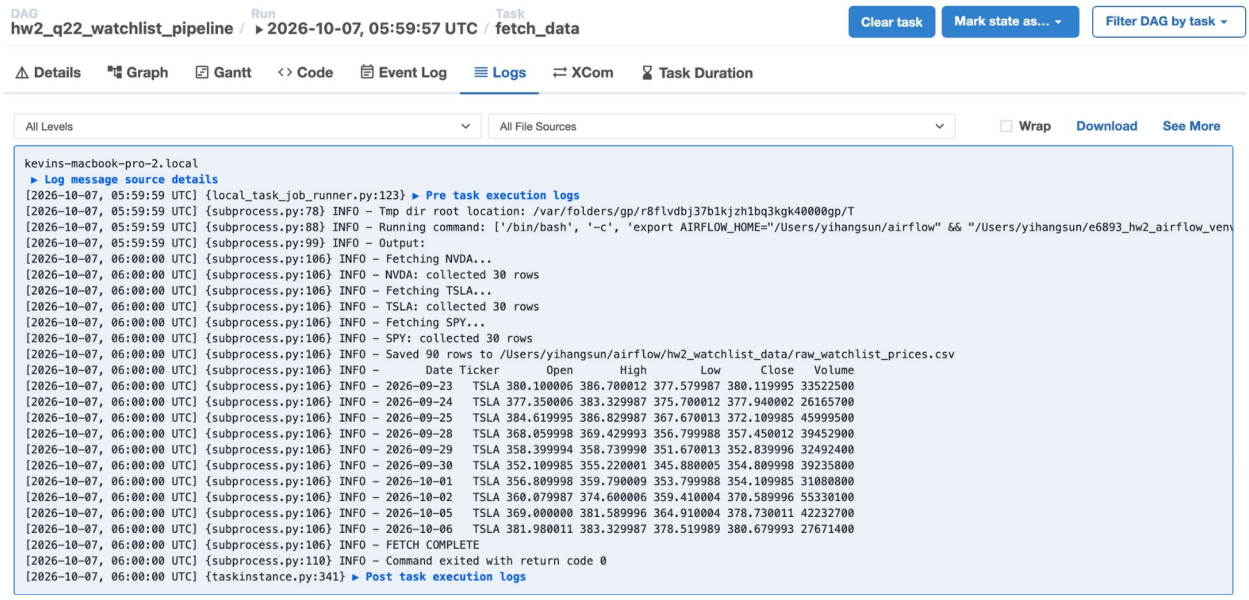


Figure 15. fetch_data task log showing 30 rows per ticker and 90 rows saved.

The analytics_summary task calculated the 30-day return, average daily price swing, and latest-day price swing for each ticker, then saved the results to watchlist_summary.csv. In this run, NVDA had a 12.2929% 30-day return, SPY had a 1.7208% return, and TSLA had an 8.6881% return. The latest-day swings were 1.8340% for NVDA, 0.4703% for SPY, and 1.2592% for TSLA.

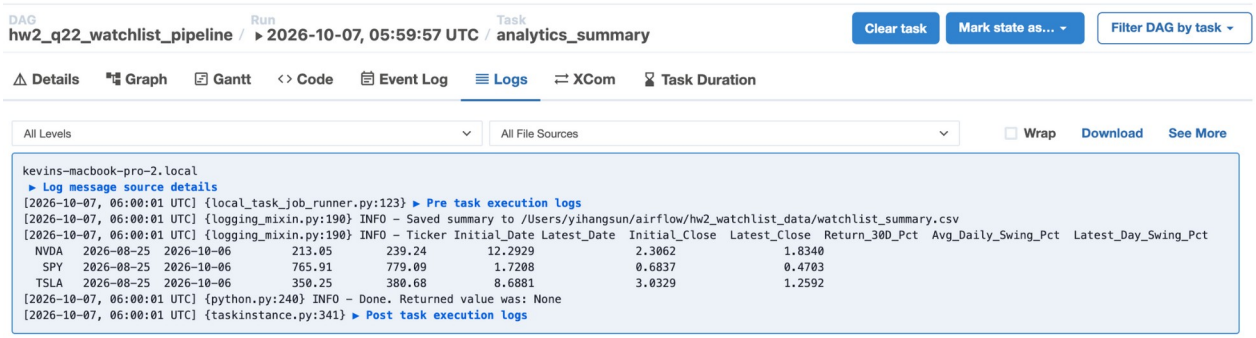


Figure 16. analytics_summary task log showing the generated watchlist summary.

The volatility_alert task compared each ticker's latest-day swing with the 3.0% threshold. None of the three latest-day swings exceeded 3.0%, so the pipeline correctly logged that all assets traded within normal volatility ranges.

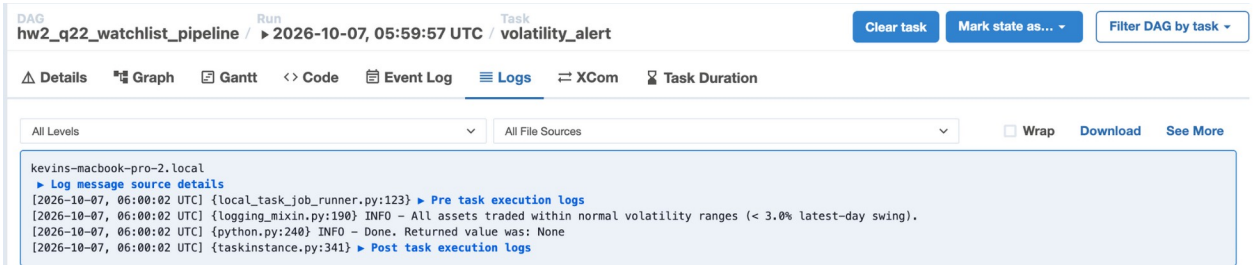


Figure 17. volatility_alert task log showing the final threshold decision.

Section 2 Summary

Section 2 demonstrated the full Airflow workflow from installation and monitoring to building and scheduling multi-task DAGs. The HelloWorld example verified the local Airflow environment and the SequentialExecutor execution model. The 19-task workflow demonstrated branching, merging, manual execution, and 30-minute scheduling. Finally, the stock watchlist pipeline connected data collection, analytics, CSV output, and a rule-based volatility check in one end-to-end DAG.

Section 3: Mini-Project

Stock Risk and Momentum Analysis
EECS E6893 Big Data Analytics - Homework 2
Fall 2026
Yihang Sun

A six-month comparison of NVDA, TSLA, and SPY using return, volatility, drawdown, moving-average momentum, and simple risk classification.

1. Project Overview

For the mini-project, I reused stock-market data because the data source was already reliable in my local environment, but I changed the analysis from the earlier Airflow task. Instead of only checking a 30-day price swing, this project compares six months of performance and risk for NVDA, TSLA, and SPY. The goal is to summarize how much each asset gained, how volatile it was, how severe its worst drawdown was, and whether its recent price trend looks bullish, neutral, or bearish.

2. Data Source

The data source is Yahoo Finance through the yfinance Python package. For each of the three assets, the program downloads six months of daily market data, including open, high, low, close, and volume. The same date range and frequency are used for all three tickers so their results can be compared directly.

3. System Function and Architecture

The program follows a simple local analytics workflow: yfinance data is downloaded first, then pandas is used to calculate return and risk features, and matplotlib is used to generate figures. The detailed daily data and the final summary are also written to CSV files. The workflow can be summarized as: Data Download -> Data Cleaning -> Feature Calculation -> Risk/Momentum Classification -> CSV Output and Visualizations.

4. Analysis Used

The analysis uses several complementary measures. Six-month return measures the total percentage change from the first closing price to the latest closing price. Annualized volatility is calculated from the standard deviation of daily returns and scaled by the square root of 252 trading days. Maximum drawdown measures the largest peak-to-trough decline during the period. I also calculate 7-day and 20-day moving averages to describe recent momentum.

For the momentum label, an asset is classified as Bullish when the latest close is above the 7-day moving average and the 7-day moving average is above the 20-day moving average. It is Bearish when the opposite ordering holds; otherwise it is Neutral. For the simple risk label, annualized volatility below 25% is Low, 25% to below 40% is Medium, and 40% or above is High.

5. Program Output

The program completed successfully and produced both detailed and summary CSV files along with three plots. The terminal summary shows that NVDA had the highest six-month return at 30.42%, TSLA had the highest annualized volatility at 49.36%, and SPY had the lowest volatility at 12.03%. The momentum labels were Bullish for NVDA and SPY and Neutral for TSLA.

```
(e6893_hw2_airflow_venv) (base) yihangsun@Kevins-MacBook-Pro-2 EECS6893_HW2_ready_v2 % python mini_project.py

Downloading NVDA...

Downloading TSLA...

Downloading SPY...

================================================================================
STOCK RISK AND MOMENTUM SUMMARY
================================================================================
Ticker  Initial_Close  Latest_Close  Six_Month_Return_Pct  Annualized_Volatility_Pct  Max_Drawdown_Pct   MA_7  MA_20 Momentum Risk_Level
  NVDA         182.08        237.47                 30.42                      38.79            -19.40 233.72 225.58  Bullish     Medium
  TSLA         343.25        377.81                 10.07                      49.36            -33.00 367.08 367.22  Neutral       High
   SPY         676.01        777.22                 14.97                      12.03             -4.49 770.23 766.46  Bullish        Low

Saved files:
mini_project_output/stock_analysis_data.csv
mini_project_output/stock_risk_momentum_summary.csv
mini_project_output/cumulative_return.png
mini_project_output/rolling_volatility.png
mini_project_output/normalized_price.png

MINI-PROJECT COMPLETE
```

Figure 1. Terminal output showing the completed mini-project and summary metrics.

6. Summary CSV

The summary CSV contains the initial and latest closing prices, six-month return, annualized volatility, maximum drawdown, 7-day moving average, 20-day moving average, momentum label, and risk level for each ticker. The results show a clear return-risk tradeoff: NVDA produced the strongest return with medium risk, TSLA had the highest risk and deepest drawdown, and SPY was the most stable asset in the comparison.

A	B	C	D	E	F	G	H	I	J
Ticker	Initial_Clos	Latest_Clo	Six_Month	Annualizec	Max_Draw	MA_7	MA_20	Momentun	Risk_Level
NVDA	182.08	237.47	30.42	38.79	-19.4	233.72	225.58	Bullish	Medium
TSLA	343.25	377.81	10.07	49.36	-33	367.08	367.22	Neutral	High
SPY	676.01	777.22	14.97	12.03	-4.49	770.23	766.46	Bullish	Low

Figure 2. Generated stock_risk_momentum_summary.csv.

7. Six-Month Cumulative Return

The cumulative-return plot shows how the three assets evolved over the six-month period. NVDA and TSLA both rose sharply early in the period, but their paths were much more volatile than SPY. TSLA experienced a large decline around late July before recovering, while NVDA finished the period with the strongest cumulative gain. SPY followed a smoother upward path and ended with a more moderate return.

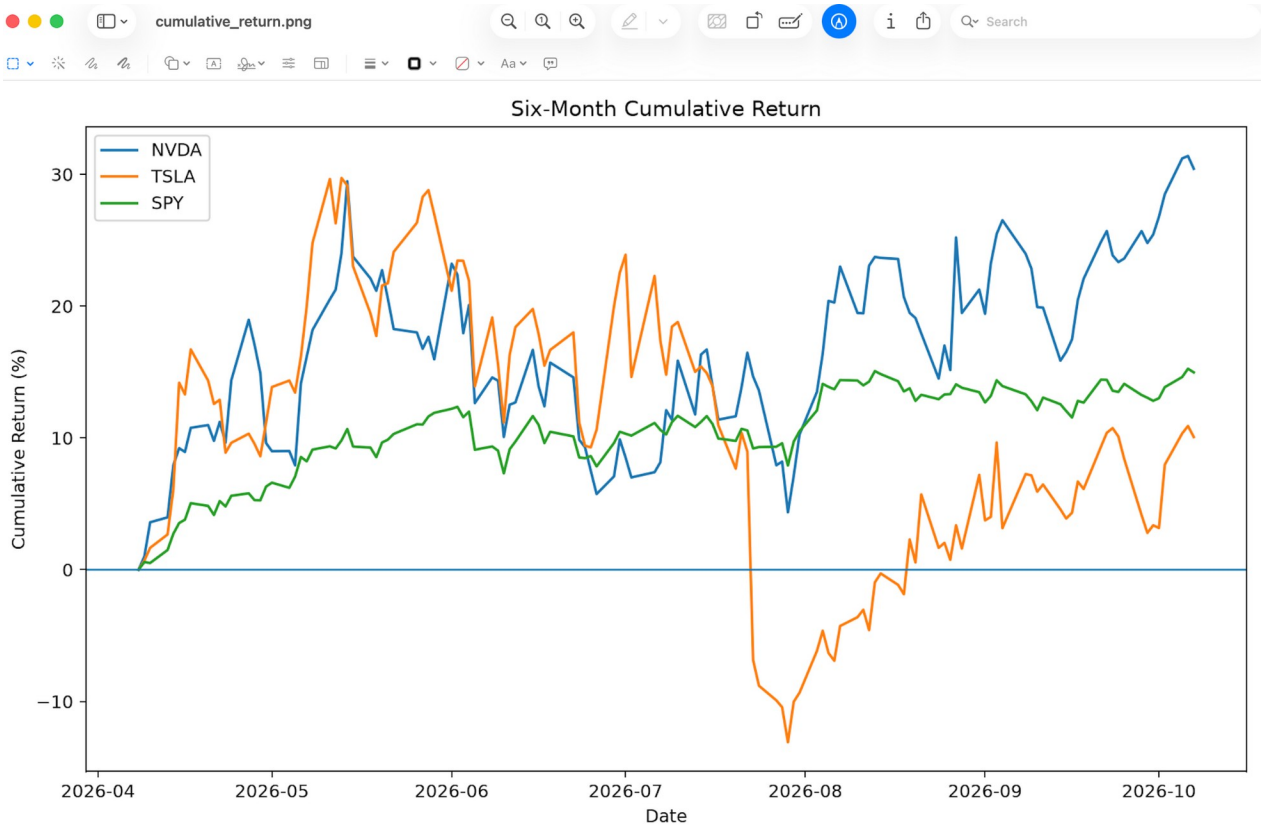


Figure 3. Six-month cumulative return for NVDA, TSLA, and SPY.

8. Rolling Volatility

The 20-day rolling annualized volatility plot highlights the difference in risk across the three assets. TSLA was the most volatile for much of the period and reached the highest volatility spike. NVDA also showed substantially more volatility than SPY. SPY remained comparatively stable, which is consistent with its Low risk classification in the summary output.



Figure 4. 20-day rolling annualized volatility.

9. Normalized Price Performance

To compare assets with very different dollar prices, I normalized each closing-price series to 100 at the start of the period. This makes relative performance easier to compare on one chart. NVDA finished above 130, SPY finished around 115, and TSLA finished around 110, which is consistent with the six-month return values reported in the summary.

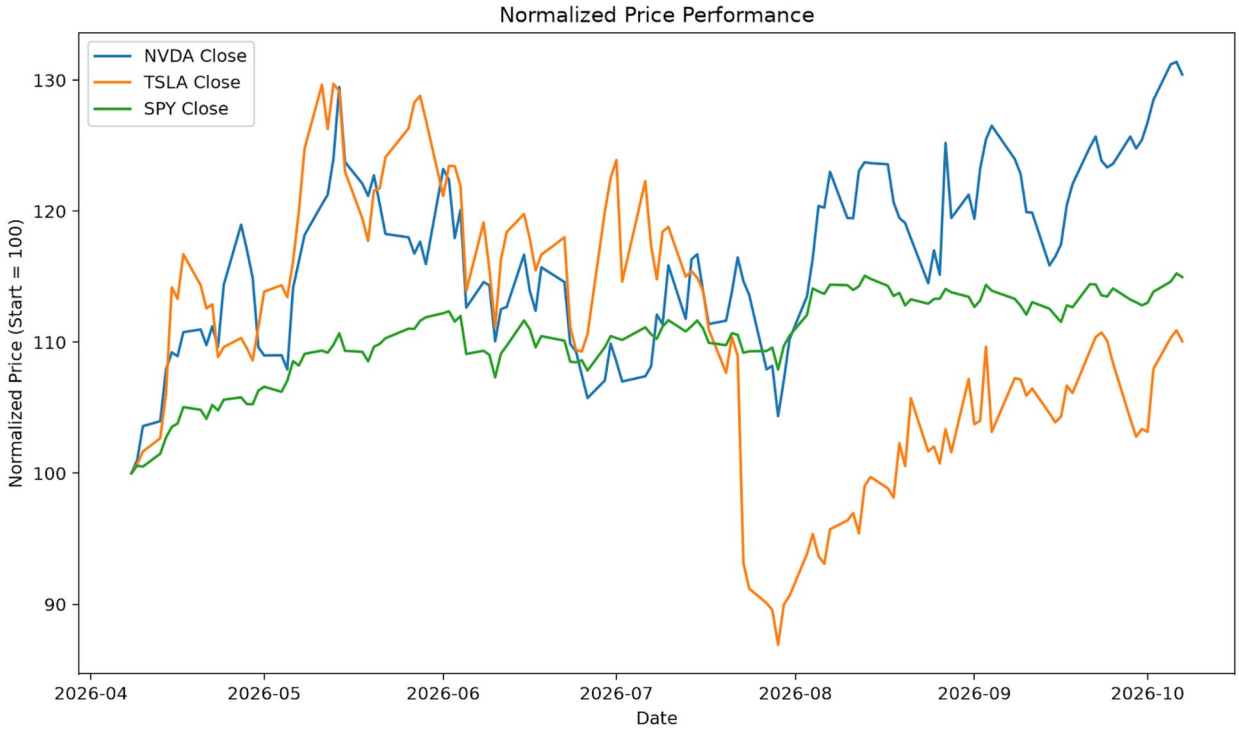


Figure 5. Normalized closing-price performance with the start value set to 100.

10. Discussion and Conclusion

The mini-project shows why return alone is not enough when comparing assets. NVDA had the best six-month return, but it also had much higher volatility and a larger drawdown than SPY. TSLA had a positive return, but it showed the highest annualized volatility and the deepest drawdown, so it received a High risk label. SPY produced a smaller return than NVDA but was much more stable, which led to a Low risk classification.

The moving-average momentum signal adds a short-term view to the longer-term return and risk measures. At the end of the sample, NVDA and SPY were Bullish because their latest prices and moving averages were ordered positively, while TSLA was Neutral. Overall, combining return, volatility, drawdown, and momentum gives a more useful picture than relying on a single metric.